

Computer Science & IT



Compiler Design



Introduction

Lecture No. 2



By- Ravindra Sir

Topics to be Covered



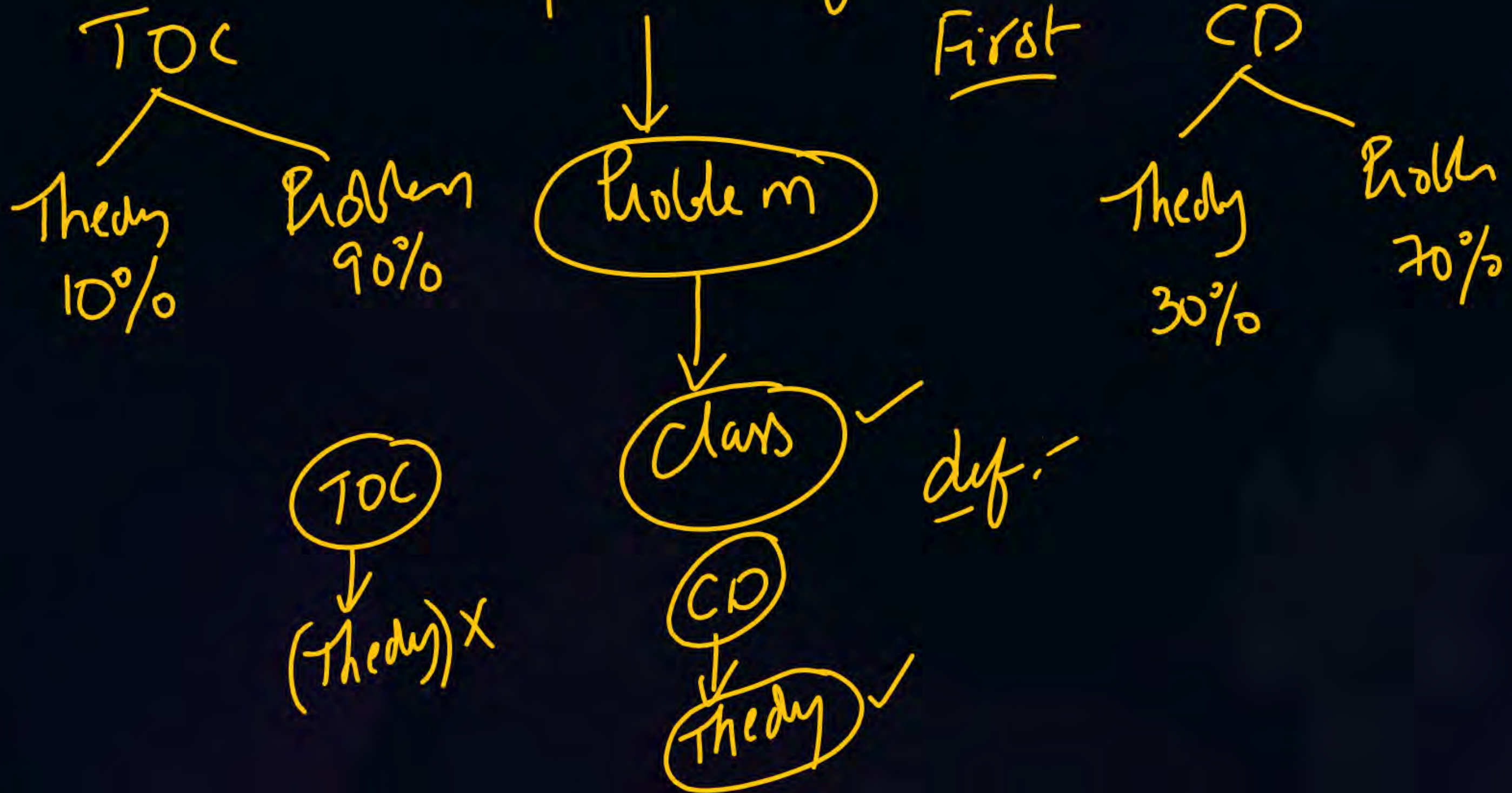
Topic

Introduction

Topic

Topic

Compiler design



15-days (2pm-5pm)

$$15 \times 3 = \underline{45 \text{ hours}}$$

c-compiler
3 months
↓
S glade

Syntax analyser:-

Syntax analyzer checks the Syntax of a given Statement. The Syntax analyzer is also called parser. The parser groups stream of tokens into a hierarchical structure called parse tree.

id + id + id



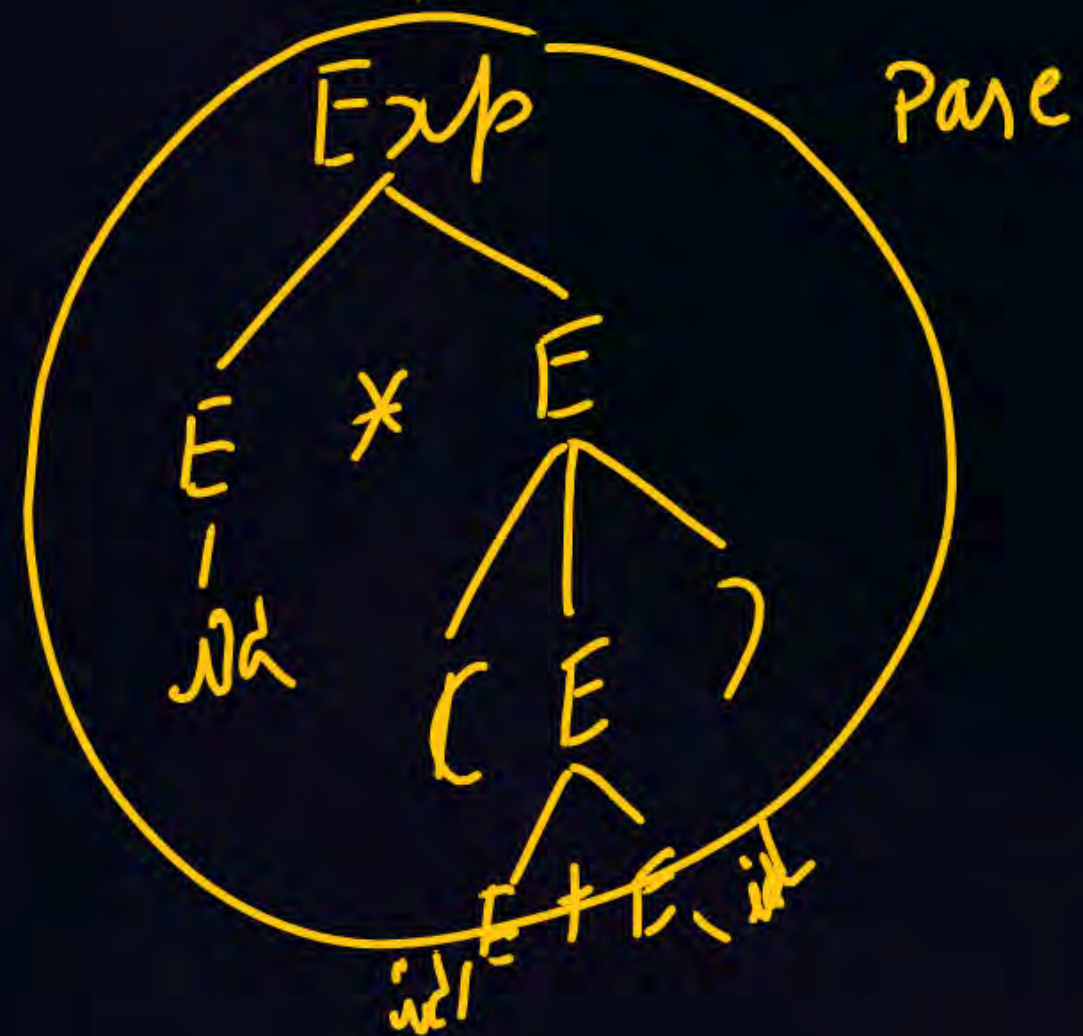
In order to check Syntax, first the Standard Syntax of all the Statements is to be known. Generally the Syntax rules for any HLL are given by CFG.



$Expr \rightarrow Expr + Expr$
 $Expr \rightarrow Expr * Expr$
 $Expr \rightarrow (Expr)$
 $Expr \rightarrow num$

CFG

$6 * (8 + 9)$



Semantic analyzer



Checks are performed to ensure that parts of a program fit together meaningfully.

Ex: $A + B$

Parser looks at binary addition and checks whether two operands are present. The presence of two operands confirms that the syntax is right. However, parser doesn't recognise the operand type.

A + B ✓

Even if 'A' is array and B is a function, parser

Says that syntax is right.

It is the Semantic analyzer that checks whether the two operands are type compatible or not.

Static type checking is the basic function of semantic analyzer. During parsing, information concerning variables

is stored in symbol table. This information is used by semantic analyzer to perform checking

Intermediate code generator :-



This phase converts the hierarchical representation of source code, that is parse tree into intermediate representation (IR) of the program.

Intermediate language should have two properties

- a) It should be simple and easy to generate
- b) It should be easy to translate to the target language

Intermediate can be represented in different way.



- 1) As a Syntax tree
- ♥ 2) Directed acyclic graph ♥
- 3) Post fix notation
- ♥ 4) 3 address code ♥

Code optimizer:-

Compiler time code optimization involves static analysis

of intermediate code to remove extra operation.

The main goal of code optimizer is to improve the efficiency. Hence it always tries to reduce the number of instructions. Once the number of instructions is reduced, the code runs faster and occupies less space.



Constant folding

$$I = 4 + j^0 - 5 \Rightarrow I = j^0 - 1$$

Constant propagation

$$\begin{array}{l|l} I=3 & \\ \downarrow & \\ J = I + 2 & \end{array} \Rightarrow \begin{array}{l} I=3 \\ J=5 \end{array}$$

Code motion & frequently reduction

```
for (i = 0; i ≤ 5000; i++)
    a = (sin x / ω x) * i;
```

loop invariant

```
t = sin x / ω x
for (i = 0; i ≤ 5000; i++)
    a = t * i;
```

Common sub exp elimination $\rightarrow$ DAG 

$$A = 100 \times (B+C)$$

$$D = 200 \times 8 + (B+C)$$

$$E = A + (B+C)$$

$$T = B+C$$

$$A = 100 \times T$$

$$B = 200 \times 8 + T$$

$$E = A + T$$

Strength reduction

$$\begin{aligned} \underline{2 \times x} &\Rightarrow \underline{x + x} \checkmark \\ \underline{2 \times x} &\Rightarrow \underline{\underline{x \ll 1}} \checkmark \end{aligned}$$

Target Code generator:-

The code generator's main function is to translate the intermediate representation of source code to the target machine code. The machine code may be executable binary code or assembly code.

To design a target code generator, the designer should have complete knowledge about the target machine

Such as

- a) Data handling
- b) machine instruction syntax
- c) variable allocation
- d) Program layout
- e) Registers
- f) Instruction set

CO

Source text:-

if ($\underline{x} \leq \underline{3}$) then $y = 2 * x$

else ($y = 5$)

Target code: assembly

10) cmp $\underline{x}$ $\underline{3}$

11) JG 15 ✓

✓ 12) mul $\underline{2}$ $\underline{x}$ Temp

✓ 13) mov Temp → y

14) jmp 16

(15) mov 5 → y

Symbol table manager and End handler:-



In addition to passing the data from one phase to another, additional information acquired during a phase may be needed by later phase. The Symbol table manager is used to store the record for each identifier, procedure encountered in the source language with relevant attributes.

int x = 10

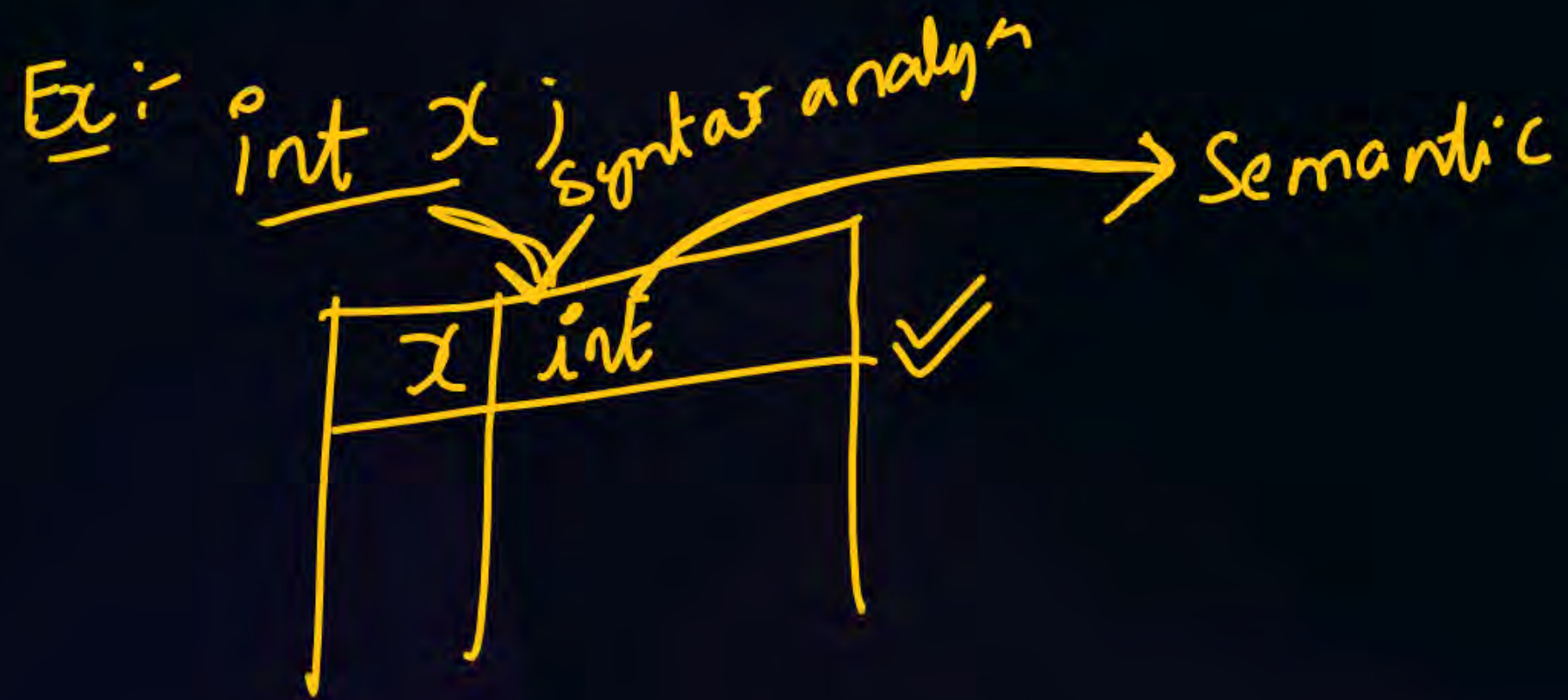
id	type	addr	token
x	int		✓

f(int a, int b, int c)

f	a	b	c	int	int	int	3 param
---	---	---	---	-----	-----	-----	---------

→ whenever any phase encounter new information, records are stored in symbol table.

→ whenever any phase requires any information, then records are retrieved from symbol table



Error handling:-

The error handler is used to report and recover from errors encountered in the source program.

A good compiler should not stop on seeing the very first error. Each phase may encounter errors, so to deal with errors and to continue further, error handlers are required in each phase.

- 1) lexical error
- 2) Syntax error
- 3) Semantic error.

a) 1:15 to 4:15

b) 1:30 to 4:15



THANK - YOU